

Tecnatura Superior en Desarrollo de Software

Módulo 06 — Validación de Modelos: `IModelValidatorHub<T>`

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

Prerequisito de lectura

Antes de avanzar con este módulo, se recomienda haber leído:

Anexo 06a — `IModelValidatorHub<T>`

Cubre: definición, analogía, interfaz, implementación concreta, comparación sin-hub/con-hub, ubicación en la arquitectura, ejemplo ejecutable en `Main()` y errores comunes.

Este módulo retoma esa base y la integra en el flujo completo de la capa `Application`: dónde vivió la responsabilidad de validación antes del hub, por qué se extrae, cómo se registra en el contenedor de DI y qué impacto tiene en el `UseCase` final.

¿Qué es la validación de modelos?

En la capa `Application`, toda operación comienza recibiendo un DTO de entrada — el *request*. Antes de que el `UseCase` pueda delegar trabajo al dominio, debe garantizar que ese request está bien formado: que los campos obligatorios existen, que las cadenas no superan sus límites, que el formato de los datos es correcto.

A esa responsabilidad la llamamos **validación de modelos**: verificar las precondiciones técnicas del DTO de entrada *antes* de que el dominio sea invocado. No es validación de reglas de negocio — eso lo hace el dominio — sino validación de formato y estructura del dato que llegó desde el exterior.

Dos niveles de validación en Clean Architecture:

Nivel 1 — Validación de modelos (Application)

¿El campo existe? ¿Tiene largo válido? ¿El email tiene formato básico?

Responsable: `IModelValidatorHub<T>` + Guards

Nivel 2 — Validación de dominio (Domain)

¿El cliente es mayor de edad? ¿El documento es único?

Responsable: Value Objects + DomainException

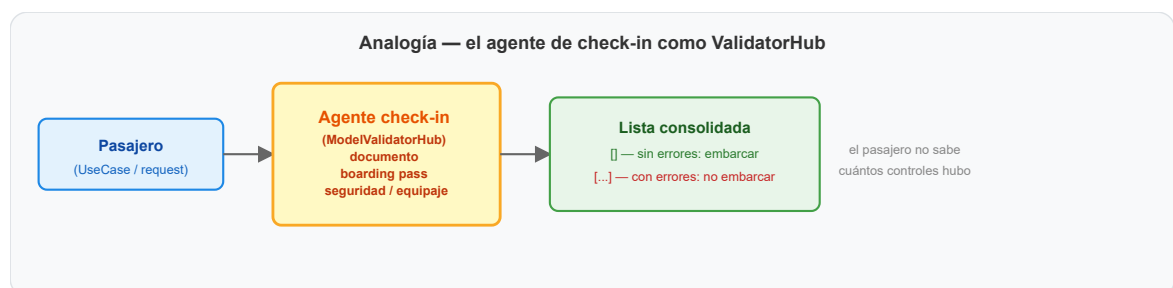
El módulo anterior introdujo los Guards — las piezas que verifican condición por condición. Este módulo introduce el mecanismo que los **orquesta**: el `ModelValidatorHub`.

Analogía del mundo real

Imaginá un control de admisión en un aeropuerto. Para embarcar, un pasajero debe pasar por distintos puestos: documento, boarding pass, seguridad, peso del equipaje. Cada puesto verifica una condición distinta.

Sin coordinación, el pasajero no sabría en qué orden hacer las colas, ni qué problema subsanar primero. Con coordinación, existe un **agente de check-in** que conoce todos los controles, los aplica en orden y entrega al final una lista consolidada de observaciones. El pasajero solo habla con ese agente.

El **ModelValidatorHub** es ese agente: el UseCase le entrega el request y recibe la lista completa de errores técnicos — sin saber cuántos Guards se ejecutaron ni en qué orden.



Características esenciales

La interfaz — el contrato genérico

`IModelValidatorHub<TModel>` es una interfaz genérica con un único método. La genéricidad permite que el mismo contrato se aplique a cualquier tipo de request sin duplicar código. El UseCase declara de qué hub depende simplemente cerrando el tipo genérico con el nombre del request concreto.

```
1 // un método, una responsabilidad – la interfaz no sabe cómo se valida
2 public interface IModelValidatorHub<TModel>
3 {
4     // devuelve todos los errores encontrados – vacío significa válido
5     IEnumerable<ValidationError> Validate(TModel model);
6 }
```

La implementación concreta — uno por request

Cada tipo de request tiene su propio hub concreto. La convención de nombre es clara: si el request es `RegisterBookRequest`, el hub es `RegisterBookValidatorHub`. Dentro, el hub instancia un `Guard` por cada campo, aplica las reglas encadenadas y combina todos los errores con `.Concat()`.

```
1 // hub concreto para RegisterBookRequest – dominio Books
2 public class RegisterBookValidatorHub
3     : IModelValidatorHub<RegisterBookRequest>
4 {
5     public IEnumerable<ValidationError> Validate(RegisterBookRequest request)
6     {
7         // un Guard por campo – cada uno acumula sus propios errores
8         var gTitle = Guard.Against(request.Title, "Title")
9             .NotNullOrEmpty().MaxLength(200);
10
11         var gAuthor = Guard.Against(request.Author, "Author")
12             .NotNullOrEmpty().MinLength(2).MaxLength(100);
13
14         var gISBN = Guard.Against(request.ISBN, "ISBN")
15             .NotNullOrEmpty().MinLength(10).MaxLength(13);
16
17         // combina todos los errores en una sola secuencia y la devuelve
18         return gTitle.Errors
19             .Concat(gAuthor.Errors)
20             .Concat(gISBN.Errors);
21 }
```

```
21     }  
22 }
```

El UseCase con hub — impacto inmediato

El hub reduce toda la lógica de validación en el UseCase a tres líneas: llamar a `Validate()`, verificar si hay errores y notificar al puerto de salida. El UseCase recupera su única responsabilidad: orquestar el flujo de negocio.

```
1  public async Task ExecuteAsync(RegisterBookRequest request)  
2  {  
3      // PASO 1 – precondiciones técnicas: el hub sabe qué validar  
4      var errors = _validator.Validate(request).ToList();  
5      if (errors.Count > 0)  
6      {  
7          // notifica todos los errores – el dominio no es invocado  
8          await _outputPort.ValidationErrorsAsync(errors);  
9          return;  
10     }  
11  
12     // PASO 2 – dominio: solo se ejecuta si el DTO es válido  
13     var book = Book.Create(request.Title, request.Author, request.ISBN);  
14     await _repository.AddAsync(book);  
15     await _outputPort.HandleSuccessAsync(...);  
16 }
```

¿Por qué se usa?

El módulo anterior mostró cómo los Guards pueden integrarse directamente en el UseCase. Esa forma funciona — pero tiene un costo que se vuelve visible a medida que el DTO crece: el UseCase acumula responsabilidades que no le corresponden.

Cada vez que el request cambia — se agrega un campo, se modifica un límite de longitud, se elimina una validación — el UseCase debe modificarse. Esto viola el **principio de responsabilidad única**: el UseCase debería cambiar solo cuando cambia la *lógica de negocio*, no cuando cambia la forma de validar la entrada.

SIN HUB — Guards en el UseCase

El UseCase mezcla orquestación de negocio con validación de entrada. Cambiar un Guard requiere tocar el UseCase.

```
1 public async Task ExecuteAsync(RegisterBookRequest r)
2 {
3     // el UseCase conoce todos los Guards — acoplamiento fuerte
4     var g1 = Guard.Against(r.Title, "Title").NotNullOrEmpty().MaxLength(20
5     0);
6     var g2 = Guard.Against(r.Author, "Author").NotNullOrEmpty().MinLength
7     (2);
8     var g3 = Guard.Against(r.ISBN, "ISBN").NotNullOrEmpty().MinLength(10);
9     var errors = g1.Errors.Concat(g2.Errors).Concat(g3.Errors).ToList();
10    if (errors.Count > 0) { await _outputPort.ValidationErrorsAsync(errors);
11    return; }
12    // ... lógica de negocio
13 }
```

CON HUB — UseCase limpio

El UseCase delega en una línea. Cambiar los Guards no requiere tocar el UseCase — solo el hub.

```
1 public async Task ExecuteAsync(RegisterBookRequest r)
2 {
3     // el UseCase no sabe qué Guards se aplican internamente
4     var errors = _validator.Validate(r).ToList();
5     if (errors.Count > 0)
6     {
7         await _outputPort.ValidationErrorsAsync(errors);
8         return;
9     }
10    // ... lógica de negocio sin ruido de validación
11 }
```

Tres beneficios del hub:

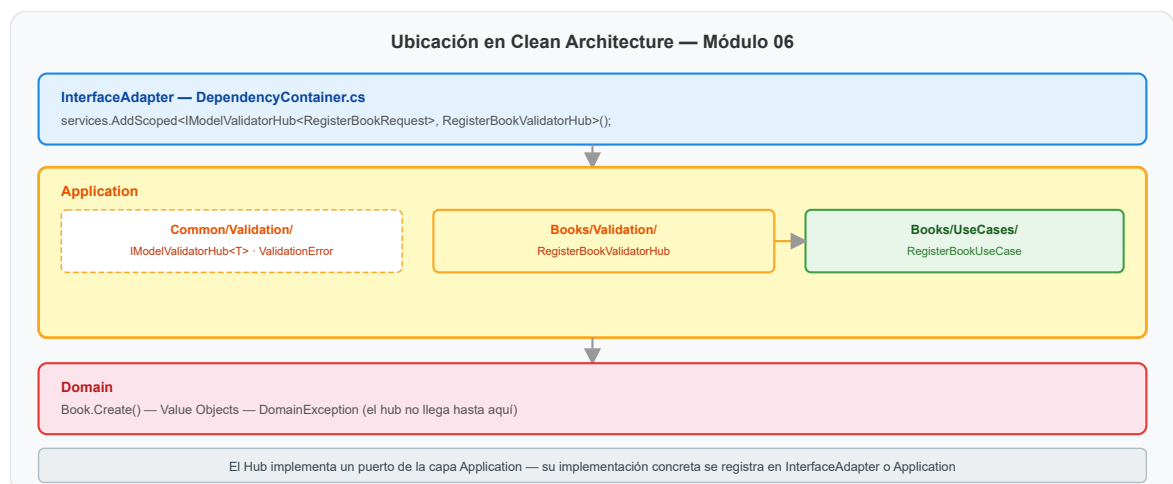
1. Separación de responsabilidades — el UseCase orquesta negocio; el hub valida el DTO.
2. Testeabilidad — el hub puede mockearse en tests del UseCase; también

puede testearse de forma aislada.

3. Cohesión — las reglas de validación de un request están todas en un único lugar.

¿Cómo encaja en la arquitectura?

El hub vive completamente dentro de la capa **Application**. La interfaz se define en `Common/Validation/` porque es compartida por todos los dominios. La implementación concreta se ubica en la carpeta del dominio al que corresponde el request.



Convención de nombres y ubicación en CommerX:

`IModelValidatorHub<T>` → `Application/Common/Validation/`

`ValidationError` → `Application/Common/Validation/`

`CreateCustomerValidatorHub` → `Application/Customers/Validation/`

Registro en DI → `Application/DependencyContainer.cs`

Registro en el contenedor de Inyección de Dependencias

El UseCase depende de `IModelValidatorHub<T>` — una interfaz. El contenedor de DI es quien decide qué clase concreta se inyecta cuando el UseCase se construye. El registro se hace en el método de extensión de la capa

Application y usa `AddScoped` porque el hub valida datos dentro del contexto de una sola operación — el mismo ciclo de vida que el UseCase.

```
1 // DependencyContainer.cs – capa Application
2 // namespace truco: usa el namespace de IServiceCollection
3 namespace Microsoft.Extensions.DependencyInjection;
4
5 public static class BookUseCasesDependencyContainer
6 {
7     public static IServiceCollection AddBookUseCases(
8         this IServiceCollection services)
9     {
10         // hub: Scoped – un hub por operación, igual que el UseCase
11         services.AddScoped<
12             IModelValidatorHub<RegisterBookRequest>,
13             RegisterBookValidatorHub>();
14
15         // UseCase: Scoped – recibe el hub por constructor
16         services.AddScoped<IRegisterBookInputPort, RegisterBookUseCase>();
17
18         return services;
19     }
20 }
```

Ciclo de vida — ¿por qué Scoped?

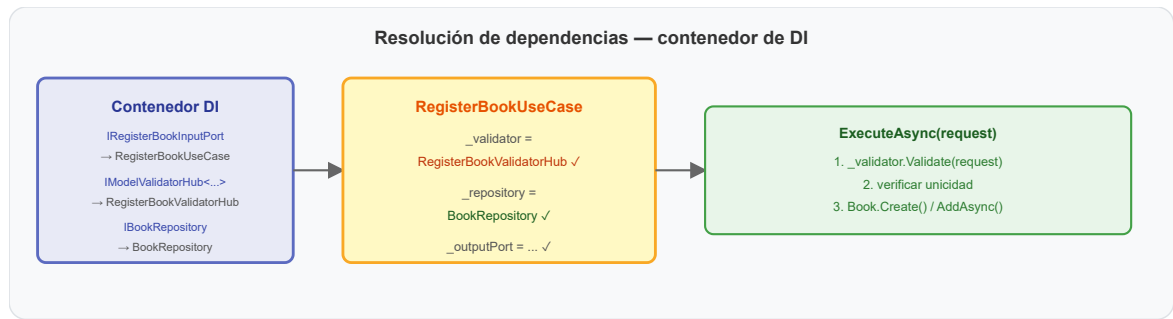
Singleton — una sola instancia para toda la aplicación: NO apto para hubs que podrían tener estado interno.

Scoped — una instancia por operación/request HTTP: CORRECTO para hubs y UseCases.

Transient — una instancia por cada inyección: posible, pero innecesario para hubs sin estado.

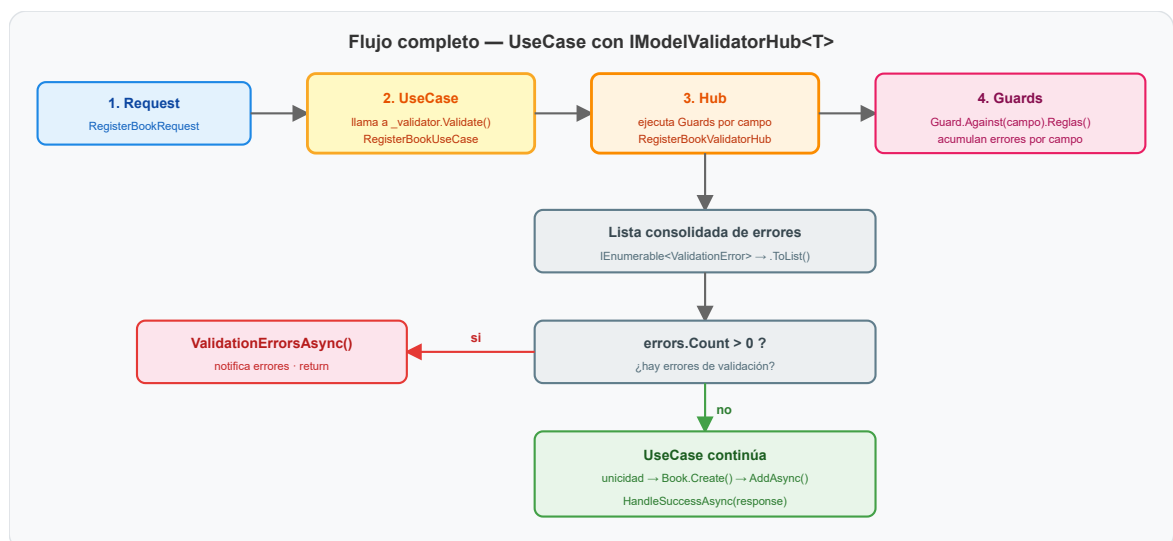
Flujo completo de resolución por DI

Cuando llega una operación, el contenedor construye el grafo de dependencias automáticamente. El UseCase declara en su constructor qué necesita; el contenedor provee cada dependencia registrada. Ninguna clase construye sus propias dependencias con `new`.



Flujo completo — request a resultado

El siguiente diagrama muestra el recorrido completo de una operación desde que el request entra al UseCase hasta que se notifica el resultado al puerto de salida — incluyendo la participación del hub y los Guards.



Marco teórico complementario

El hub como aplicación del principio de inversión de dependencias

El UseCase no crea una instancia de `RegisterBookValidatorHub` directamente. Declara en su constructor que necesita una implementación de `IModelValidatorHub<RegisterBookRequest>` — y el contenedor de DI la provee. Esto es **inversión de dependencias**: el módulo de alto nivel (UseCase) depende de una abstracción, no de un detalle de implementación.

La consecuencia práctica es que en un test unitario se puede sustituir el hub por un objeto controlado — un mock — que devuelva los errores que el test necesita para verificar el comportamiento del UseCase ante entradas inválidas. El UseCase no cambia; solo cambia la implementación inyectada.

El hub como aplicación del principio de responsabilidad única

El UseCase tiene una sola razón para cambiar: cuando cambia la *lógica de negocio* de la operación. El hub tiene otra sola razón para cambiar: cuando cambian las *precondiciones técnicas* del DTO de entrada. Al separarlos, se respeta SRP y ambas clases se vuelven más simples, más pequeñas y más fáciles de razonar.

Resumen de responsabilidades en la capa Application:

UseCase → orquesta el flujo de negocio: validar, buscar, crear, persistir, notificar.

Hub → valida las precondiciones técnicas del DTO: formato, longitud, obligatoriedad.

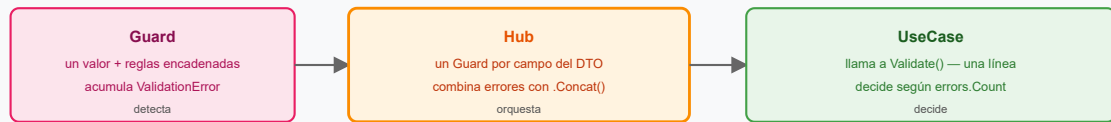
Guard → evalúa una condición específica sobre un valor y acumula errores.

ValidationError → estructura de dato que describe un error: campo + mensaje.

Relación entre Guard y Hub — dos capas de responsabilidad

El Guard opera sobre un valor individual con reglas encadenadas. El Hub coordina múltiples Guards — uno por campo del DTO — y combina sus resultados. Son responsabilidades complementarias: el Guard *detecta*, el Hub *orquesta*, el UseCase *decide*.

Guard, Hub y UseCase — capas de responsabilidad



Errores comunes

Error 1 — registrar el hub como Singleton

Si el hub se registra como `Singleton`, existe una única instancia para toda la vida de la aplicación. Si en el futuro el hub necesita una dependencia con ciclo de vida `Scoped` — como un repositorio — se producirá un error de resolución en tiempo de ejecución. Siempre registrar como `Scoped`.

ERROR COMUN

```
1 // MAL: Singleton no es compatible con dependencias Scoped
2 services.AddSingleton<IModelValidatorHub<RegisterBookRequest>, RegisterBookValidatorHub>();
```

BUENA PRACTICA

```
1 // BIEN: Scoped — un hub por operación
2 services.AddScoped<IModelValidatorHub<RegisterBookRequest>, RegisterBookValidatorHub>();
```

Error 2 — omitir el registro del hub

Si el hub se implementa pero no se registra en el contenedor de DI, el UseCase no puede resolverse y la aplicación lanza una excepción en tiempo de ejecución al intentar construir el UseCase. El mensaje típico es: *"Unable to resolve service for type IModelValidatorHub..."*. Cada hub implementado debe tener su correspondiente línea en `DependencyContainer.cs`.

Error 3 — llamar a ToList() dentro del hub

El hub devuelve `IEnumerable<ValidationError>` — una secuencia perezosa. El UseCase llama a `.ToList()` sobre el resultado para poder verificar `Count`. Si el hub ya llama a `.ToList()` internamente y el UseCase también lo hace, la evaluación se duplica. El contrato correcto es: el hub devuelve la secuencia sin materializar; el UseCase la materializa una vez cuando necesita el conteo.

ERROR COMUN

```
1 // MAL: el hub materializa – el UseCase vuelve a materializar
2 return g1.Errors.Concat(g2.Errors).ToList(); // en el hub
3 var errors = _validator.Validate(r).ToList(); // en el UseCase – duplicado
```

BUENA PRACTICA

```
1 // el hub devuelve la secuencia sin materializar
2 return g1.Errors.Concat(g2.Errors);
3 // el UseCase materializa una sola vez
4 var errors = _validator.Validate(r).ToList();
```

Qué NO hacer

No incluir validación de dominio en el hub

Las precondiciones que verifica el hub son técnicas: largo mínimo, campo obligatorio, formato básico. Las reglas de dominio — edad mínima, unicidad, consistencia entre campos — pertenecen al dominio o al UseCase. Mezclarlas en el hub viola la separación de capas y hace que la capa Application conozca detalles del dominio.

QUE NO HACER

Regla de negocio (mayor de edad) en el hub — no le corresponde.

```
1 // MAL: regla de dominio en el hub
2 if (request.BirthDate.AddYears(18) > DateTime.Today)
3     yield return new ValidationError("BirthDate", "Debe ser mayor de edad.");
```

BUENA PRACTICA

El hub verifica formato; el dominio aplica la regla de negocio en Create().

```
1 // hub: solo precondition técnica — ¿la fecha existe?
2 var g = Guard.Against(request.BirthDate.ToString(), "BirthDate").NotNullOrEmpty();
3 // la regla de mayor de edad: Value Object BirthDate en el Domain
```

Fronteras claras entre validación técnica y de dominio:

Hub (Application) → "¿El campo existe y tiene formato correcto?"

Value Object (Domain) → "¿El valor cumple las reglas del negocio?"

UseCase (Application) → "¿El dato ya existe en el sistema (unicidad)?"